MVAM: A Multi-View Attention Model for Steering Angle Prediction

Evaluation Report

MohammadHossein Shamsipour Aida Khaleghi

2025

Abstract

This report describes MVAM, a small model that predicts a car's steering angle from three camera views (center, left, right) recorded in a driving simulator. The model uses a shared ResMLP backbone to extract features from each view, a multi-head attention layer to combine the three views, and a small MLP to output the final steering angle. We explain the data, the architecture, and the training setup, and then walk through the evaluation results, including where the model does well and where it still struggles.

1 Introduction

Predicting a steering angle from camera images is a common first project in autonomous driving research, mainly because it is easy to set up with a driving simulator and gives a clear, numeric target to optimize. The interesting part of this project is not the steering task itself but how to combine information from more than one camera. A human driver checks mirrors and peripheral vision, not just the road straight ahead; we wanted the model to do something similar by learning how much to trust the center, left, and right camera at each moment, instead of just treating the three images as one wide picture.

This is done with an attention layer placed on top of a shared image backbone. The rest of this report covers the dataset, the model, the training setup, and a detailed look at the evaluation numbers.

2 Dataset and Preprocessing

The data comes from the `training-car` dataset on Kaggle, which is based on the well-known Udacity self-driving-car simulator recordings. Each row in `driving_log.csv` points to a center, left, and right image captured at the same time, together with the steering angle, throttle, brake, and speed recorded at that instant. We download the dataset directly through `kagglehub` at the start of the training notebook.

2.1 Steering angle distribution

Figure 1 shows the distribution of steering angles in the dataset. As is typical for driving data, most of the recording is close to straight-line driving, so the distribution is heavily concentrated around zero, with a long but sparse tail toward sharper left and right turns. This is a normal property of

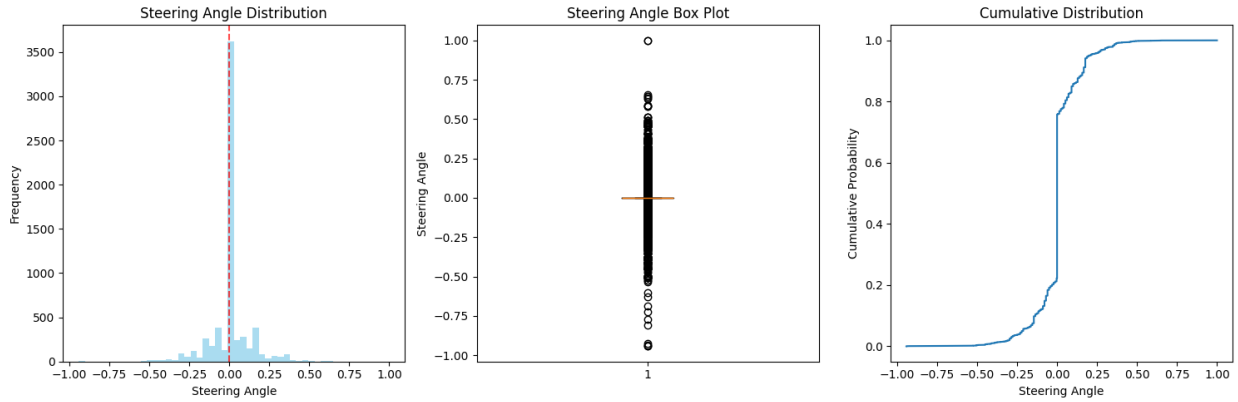

Figure 1: Steering angle histogram, box plot, and cumulative distribution over the full dataset.

driving logs, but it is also a problem for training: a model can get a deceptively low error simply by predicting values close to zero all the time, without actually learning to steer.

To reduce this problem, we:

- normalize the steering angle before training (min-max, z-score, or tanh scaling; min-max was used for the results in this report),
- build a weighted sampler that bins the steering angles and gives rarer bins (sharper turns) a higher chance of being sampled during training,
- apply light augmentation to training images only: random color jitter, a random horizontal flip, and a small amount of Gaussian noise.

3 Model Architecture

3.1 Backbone

Each of the three camera images is passed through the same ResMLP backbone (`resmlp_12_224` from the `timm` library, pretrained on ImageNet). Because the backbone is shared, it learns a single, general-purpose way of describing a road image rather than three separate ones, which keeps the parameter count down and lets features from the three views live in the same space, ready to be compared by the attention layer. To avoid destroying the pretrained features early in training, we freeze the first 60% of the backbone's layers and fine-tune only the remaining layers.

3.2 Cross-view attention fusion

After the backbone, we have three feature vectors: one for the center, left, and right image. These are stacked and passed through a multi-head scaled dot-product attention layer:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V,$$

where Q, K, and V are learned projections of the three stacked feature vectors, and d_k is the dimension of each attention head. Each view can attend both to itself and to the other two views, and the attention weights are learned during training rather than fixed in advance. The three attended feature vectors are then averaged into a single fused vector.

3.3 Regression head

The fused feature vector goes through a small bottleneck MLP ($512 \to 256 \to 128 \to 1$), with ReLU activations, dropout, and batch normalization between layers, producing the final steering angle prediction.

3.4 Loss function and training

We use a custom loss combining the mean squared error with a small smoothness penalty on the size of the prediction:

$$\mathcal{L} = w_{\text{mse}} \cdot \text{MSE}(\hat{y}, y) + w_{\text{smooth}} \cdot \frac{1}{n} \sum_{i=1}^{n} |\hat{y}_i|,$$

with $w_{\text{mse}} = 1.0$ and $w_{\text{smooth}} = 0.1$. The smoothness term discourages large, jerky steering predictions, which matters for passenger comfort even if it is not directly measured by MSE. The model is trained with AdamW, a cosine annealing learning rate schedule, and gradient clipping at a norm of 1.0, for up to 50 epochs. The checkpoint with the lowest validation loss is kept as the final model.

4 Evaluation Metrics

We report standard regression metrics as well as a few metrics specific to steering prediction.

4.1 Regression metrics

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{1}$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i| \tag{2}$$

$$\text{RMSE} = \sqrt{\text{MSE}} \tag{3}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{n} (y_i - \bar{y})^2} \tag{4}$$

4.2 Directional accuracy

The share of predictions that get the sign (left vs. right) of the steering angle correct:

$$\text{Directional Accuracy} = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}[\text{sign}(y_i) = \text{sign}(\hat{y}_i)].$$

4.3 Angular accuracy

The share of predictions within a given error threshold τ:

$$\text{Angular Accuracy}_\tau = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}[|y_i - \hat{y}_i| \leq \tau].$$

We report this at $\tau \in \{0.05, 0.10, 0.15, 0.20\}$ radians (about 2.9°, 5.7°, 8.6°, and 11.5°).

4.4 Scenario split

We separate the validation set into straight driving ($|y_i| \leq 0.05$ rad) and turning ($|y_i| > 0.05$ rad), and further split turns into left ($y_i < -0.05$) and right ($y_i > 0.05$), to see whether the model performs differently across these situations.

5 Results

All numbers below come from the same validation run and are also stored in `evaluation_results.json`. The validation set has 1,608 samples: 1,015 straight-driving and 593 turning (300 left, 293 right).

Metric	Value
MSE	0.0100
MAE	0.0647
RMSE	0.0999
R^2	0.365
Directional accuracy	36.1%
Angular accuracy	
Within 0.05 rad	62.7%
Within 0.10 rad	78.5%
Within 0.15 rad	89.0%
Within 0.20 rad	93.8%
By driving scenario	
MAE, straight driving	0.0335
RMSE, straight driving	0.0516
MAE, turns	0.1183
RMSE, turns	0.1500
MAE, left turns	0.1149
MAE, right turns	0.1217

Table 1: Core regression and steering-specific metrics on the validation set.

Statistic	Value
Median absolute error (P50)	0.0325
90th percentile error (P90)	0.1548
95th percentile error (P95)	0.2174
99th percentile error (P99)	0.3612
Maximum absolute error	0.6712
Large-error rate ($\|\text{err}\| > 0.3$)	4.3%
Std. dev. of predictions	0.0666
Std. dev. of true values	0.1253
Mean bias	0.0116

Table 2: Error distribution and bias statistics on the validation set.

Figure 2 shows the full evaluation grid, and Figure 3 shows the predictions and true steering

angles over a sample sequence.

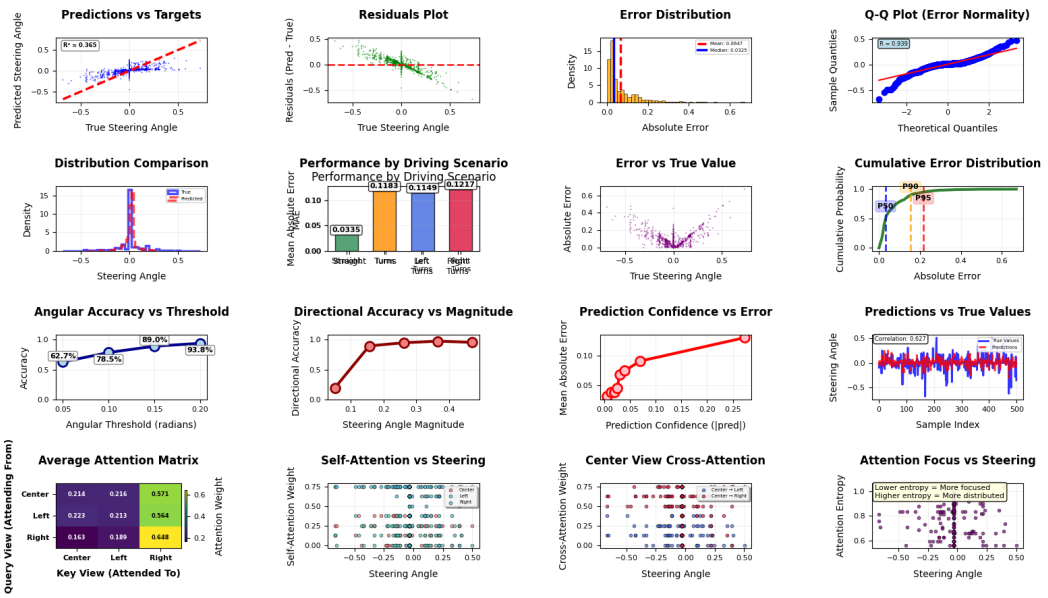

Figure 2: Full evaluation grid: predictions vs. targets, residuals, error distribution, scenario breakdown, angular accuracy, directional accuracy vs. magnitude, and the attention analysis panels.

5.1 Attention pattern

One of the panels in Figure 2 shows the average attention matrix: how much each view (query, rows) attends to each view (key, columns).

Query view	to Center	to Left	to Right
Center	0.214	0.216	0.571
Left	0.223	0.213	0.564
Right	0.163	0.189	0.648

Table 3: Average attention weights, averaged over the validation set.

The clearest pattern here is that all three views end up attending mostly to the *right* camera (0.57 to 0.65), more than to themselves or to the left camera. We did not design the model to prefer any particular view, so this looks like something the network converged to on its own, possibly related to how the simulator track is laid out, or simply a local optimum reached during training. It would be worth checking whether this pattern holds on a different track or a different random seed before drawing strong conclusions from it.

6 Discussion

Taken together, the results describe a model that has learned the overall shape of the steering task but is still fairly cautious. A few points stand out.

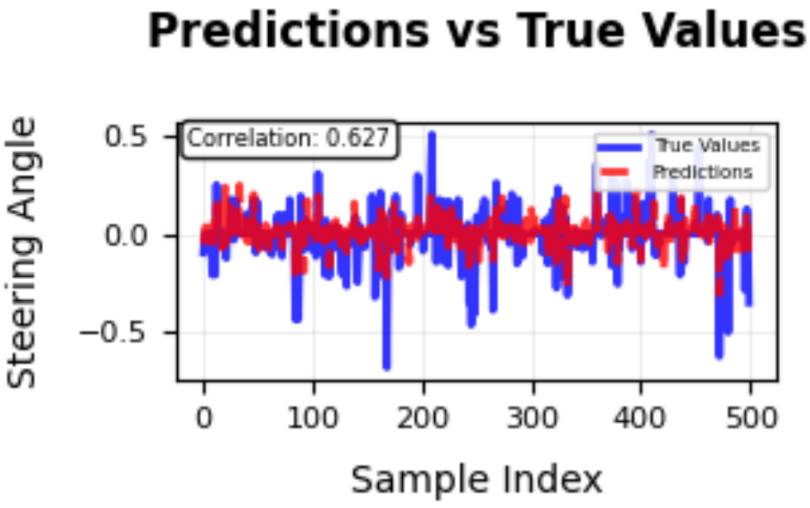

Figure 3: Predicted vs. true steering angle over a sample sequence of 500 points (correlation ≈ 0.63).

The model under-reacts to sharp turns. The standard deviation of its predictions (0.067) is about half that of the true steering angles (0.125), and the MAE for turns (0.118) is roughly 3.5 times higher than for straight driving (0.034). Figure 3 shows this directly: the prediction curve (red) follows the general shape of the true curve (blue) but visibly lags and flattens out around the sharpest peaks. Some of this is expected, since the smoothness term in the loss actively discourages large predictions, and turning is also the rarer, harder case in the data despite the weighted sampler.

Directional accuracy is misleading here. At 36.1%, it looks like the model gets the turning direction wrong most of the time, which would be a serious problem in a real driving system. But almost two thirds of the true steering angles are within 0.05 rad of zero (Figure 1), where the "correct sign" is not very meaningful — a true value of $+0.001$ and a prediction of -0.001 are both, in practice, "drive straight," yet they count as a directional miss. The angular accuracy numbers in Table 1 (63% within 0.05 rad, rising to 94% within 0.20 rad) give a much fairer picture of prediction quality, and we would recommend reporting angular accuracy alongside, or instead of, directional accuracy for this kind of steering data.

Left and right turns are handled about equally well. MAE for left turns (0.1149) and right turns (0.1217) are close, with only a small mean bias (0.0116) toward positive (right) values. This suggests the balanced sampler is doing its job in preventing the model from favoring one direction.

7 Limitations and Future Work

The main limitation is the model's conservative behavior on sharp turns, which is the clearest place to improve. A few directions that could help:

- lowering the smoothness weight in the loss, or replacing it with a term that only penalizes unrealistically large steering changes between consecutive frames, rather than the raw prediction

magnitude;

- adding a small temporal model (e.g. a GRU or 1D convolution over a short window of frames) so the model can use recent steering history instead of treating every frame independently;
- collecting or sampling more turning examples, since turns still make up only 37% of the validation data even after balanced sampling during training.

It is also worth replacing directional accuracy with a metric that accounts for the near-zero cluster of steering angles, and re-checking the attention pattern in Table 3 on a different track or training run to see if the right-camera preference is a real, repeatable effect or just something specific to this run.

Finally, this project was trained and evaluated entirely on simulator data, so the results describe how the model behaves in that simulator, not how it would behave on a real road.

8 Conclusion

MVAM combines a shared ResMLP backbone with a learned attention layer to fuse three camera views for steering angle prediction. On the validation set, it reaches an MAE of 0.065 and explains about 36% of the variance in steering angle ($R^2 = 0.365$), with strong accuracy for near-straight driving (63% of predictions within 0.05 rad) but more error during sharp turns. The attention analysis shows a consistent, possibly unintended, preference for the right camera view. Together, these results point to a model that has learned a reasonable, if cautious, driving policy, with turn handling and directional metric choice being the clearest areas for future work.

References

- Touvron, H., Bojanowski, P., Caron, M., et al. *ResMLP: Feedforward Networks for Image Classification with Data-efficient Training.* arXiv:2105.03404, 2021.
- Wightman, R. *PyTorch Image Models (timm).* GitHub repository, 2019. `https://github.com/huggingface/pytorch-image-models`
- Vaswani, A., Shazeer, N., Parmar, N., et al. *Attention Is All You Need.* NeurIPS, 2017.
- `roydatascience/training-car`, Kaggle dataset, based on the Udacity self-driving-car simulator.